Website build tracker: a Google Doc spec

A reusable spec for the shared progress document on a website build. Written to be dropped into another project's instructions so the same doc gets built the same way.

---

1. What it is for

One document that answers three questions at a glance:

1. **What exists?** Every page, and whether it has been built.
2. **What is being worked on, and what has been signed off?** Not the same question as 1.
3. **What is blocked, and on whom?**

It is a **review tool**, not a status report. The test of whether it works: the reviewer opens it, finds the next thing to look at without asking, and has somewhere to put feedback that will not get lost.

The mistake to avoid

The first version of this doc used one status column with values like Done / In progress / Open. That collapsed three independent facts into one word:

- the page is built and passes the automated checks (an engineering fact)
- the page's design and copy have been finalised by the person who owns it
- the client has signed it off

"Done" read as all three. Nothing was approved, and the doc said otherwise. **Keep "built" and "approved" as separate states from the start.** Every page begins at Built, and Built means "nobody has looked at it yet".

---

2. Structure

Six sections, in this order. The order matters: the thing the reviewer uses daily is section 1, and everything below it is reference.

#	Section	Holds
—	Title, one-line purpose, how-to-use paragraph	Who does what, and what the statuses mean in practice
—	Status key	The vocabulary and its colours
—	Environments	Staging URL, production URL, final domain, and what each costs to deploy
1	**Site structure**	The per-page tracker. This is the working surface.
2	Recently completed	What shipped, with the measurement that proved it
3	Open decisions	Items waiting on a decision, with the recommendation attached
4	Needs client input	Blocked on an asset or a sign-off from the client
5	Known deviations, accepted on purpose	Things that look like mistakes but are not
6	Before launch	The pre-launch checklist

Sections 4 and 5 exist to stop the same conversation happening twice. Section 5 in particular: write down *why* a rule was broken, or someone will "fix" it later.

---

3. The per-page tracker

Four columns:

Page	Path	Status	Feedback and notes

- **Page** — the human name, matching the page's own h1 or nav label.
- **Path** — the URL path. The reviewer clicks it against the staging host.
- **Status** — one value from the key. The cell is colour-filled.
- **Feedback and notes** — free text. The reviewer types here, or comments on the row.

Grouping rule

One row per page **that has unique copy**. Group only pages that share a template and will be reviewed once:

- templated family pages (e.g. 10 market pages from one template) → one row
- templated directory pages (e.g. 19 specialty pages) → one row
- generated archive pages (e.g. a page per year) → one row
- long-tail content (e.g. 1,000 migrated articles) → one row

On a 1,100-page site this produced ~40 rows. Listing every page is unusable; grouping by site section is useless, because feedback about one page has nowhere to land.

Ordering

By review priority, not by site structure. Homepage first, then the pages that carry traffic or revenue, then the long tail. The reviewer works top-down.

---

4. Status vocabulary and colours

Six states. Green means finished, whatever kind of thing it is.

Status	Meaning	Fill	Hex
Built	Page exists and passes the automated checks. No design or content review yet.	none	—
In review	Being worked on now.	orange	`#f6b26b`
Changes requested	Feedback captured, not yet applied.	yellow	`#ffe599`
Approved	Signed off. Do not change without asking.	green	`#b6d7a8`
Needs you	Blocked on a decision or an asset from the client.	red	`#ea9999`
Open / **Deferred**	Agreed or parked. Not started.	grey	`#d9d9d9`

These are Google's standard light palette, so black body text stays legible on all of them and they match what a person gets from the Docs fill picker.

Colour every status cell in the document, not just the tracker. Sections 3 and 6 have status columns too. A colour scheme that stops halfway down reads as broken. If those sections use a different vocabulary (Done / Open / Deferred), map them onto the same colours and add the extra rows to the key so the mapping is documented.

Green covers both Approved (a page) and Done (a work item). One colour, one meaning: finished.

---

5. How the two people use it

- **Builder:** moves a row to In review when starting it. Moves it to Approved only when the reviewer says so. Updates sections 2, 3 and 6 as work lands.
- **Reviewer:** opens the first row that is not green. Clicks the path. Leaves feedback in the Notes cell, or comments on the row, then sets the status to Changes requested.

Notes cell vs comment: the cell is for a standing requirement that should persist ("form not wired up yet"). A comment is for a conversation, and it resolves. Use both.

The how-to-use paragraph at the top should name who is doing what. Write it as prose, not a legend — it is the only part a newcomer reads.

---

6. Building and maintaining it

Creating

Write the whole document as **HTML** and upload it through Google Drive's file-create with `contentMimeType: "text/html"`. Drive converts `<h1>`, `<h2>`, `<p>`, `<table>`, `<b>` and `<a>` into native Docs formatting. This is far faster and less error-prone than building a doc with Docs API edit requests.

Set `favicon`-equivalent details later; the title you pass becomes the file name.

Updating

A Drive connector alone **cannot edit a doc after creating it**. You need the Google Docs connector as well. Without it, every change means a new file and a new link, which defeats the purpose. Check this before promising a living document.

With it, the rules that actually bite:

1. **Indexes are UTF-16 offsets from the start of the tab**, and every insert or delete shifts everything after it. Order the requests in a batch **highest index first**.
2. **Indexes go stale after any write.** Read again before every batch that uses them.
3. **Guard every batch** with `writeControl.requiredRevisionId` from a read taken after your last write. If a human is editing the doc at the same time, the batch is rejected rather than landing in the wrong place. This happens in practice — it happened here. Re-read and rebuild; never force.
4. **Use `replaceAllText` where the find string is unique.** It needs no indexes and no read, and it is the safest way to change a status word or a sentence.
5. **To rewrite a table cell**, delete `[start, end - 1]` and insert at `start`. Keeping the trailing newline keeps the paragraph and its style.
6. **To replace a whole table**, delete exactly `[table startIndex, table endIndex]`, then insert the new table at the preceding paragraph's `endIndex - 1`. Two separate batches, with a read in between.
7. **Cell fills** use `updateTableCellStyle` with `rgbColor` values from 0 to 1, over a `tableRange` built from the table's start index. Colour requests do not shift indexes, so they can all go in one batch — but do them **before** any text insert in the same region, or re-read first.
8. **Inserted text inherits the style of the text before it.** Inserting after a bold run makes the whole insert bold. Reset it explicitly with `updateTextStyle`.

Keeping it honest

The tracker goes stale faster than the site does. Two habits:

- When a decision lands, update section 3 **in the same session**, not later.
- Re-derive the page list from the build output rather than typing it. On this project that was a script over the built HTML, reading each page's path and `h1`. Doing that surfaced two real content bugs that nobody had noticed: an acronym lowercased by a template (`ENT` → `eNT`) and a heading that read correctly on screen but extracted as `Revenue.Confidence`.

That last point is the quiet argument for building the tracker from the build rather than by hand: the inventory pass is also an audit.